# 코드 전체 설명서 — 운영 인시던트 대응 에이전트

> 표시: 🟦 **원본** (베이스 그대로) · 🟩 **신규** (이번에 새로 작성) · 🟨 **수정** (베이스를 고침) 읽는 순서 추천: 1장(큰 그림) → 2장(핵심 개념) → 3장(요청 하나 따라가기) → 4장(파일별 상세)

---

## 1. 큰 그림

### 1-1. 프로세스가 몇 개 떠 있나

```
[브라우저 web_chat.html]
        │ GET /chat/stream (SSE)            ← 같은 출처라 CORS 문제 없음
        ▼
[web_demo_server.py :8600]  프록시 — 받은 요청을 그대로 8100에 중계
        │
        ▼
[server.py :8100]  FastAPI ── LangGraph 그래프 실행
        │   ├─ OpenAI API (gpt-4o-mini / gpt-4o / text-embedding-3-small)  ← HTTPS
        │   ├─ session.db (SQLite)                                          ← 대화 이력
        │   └─ data/vector_index/*.json                                     ← RAG 인덱스 캐시
        │ stdio (표준입출력 파이프)
        ├──► [mcp_server.py]       "ops" 서버 — 모니터링 Tool / runbook Resource / 진단 Prompt
        └──► [mcp_server_docs.py]  "docs" 서버 — 일부러 넣은 미사용 서버(decoy)
```

- 사용자가 직접 띄우는 건 **server.py**와 **web_demo_server.py** 2개예요.
- MCP 서버 2개는 server.py가 켜질 때 **자식 프로세스로 자동 실행**되고, 서버가 꺼질 때 같이 꺼져요.

### 1-2. 폴더 지도

```
all_total_vue_student/
├── server.py                🟨 FastAPI 진입점. 서버 기동 시 의존성 조립, /chat·/chat/stream
├── web_demo_server.py       🟦 브라우저용 프록시(:8600)
├── web_chat.html            🟨 Vue 3 채팅 화면 (ReplanCheck 표시만 수정)
├── mcp_server.py            🟦 MCP "ops" 서버
├── mcp_server_docs.py       🟦 MCP "docs" decoy 서버
│
├── agent/
│   ├── graph.py             🟨 노드 연결(배선)만 — 노드 구현은 nodes/로 이동
│   ├── state.py             🟩 AgentState(그래프 공유 상태) 타입 정의
│   ├── deps.py              🟨 SupervisorDeps — 노드에 주입할 의존성 묶음 (필드 3개 추가)
│   ├── nodes/               🟩 graph.py에서 분리한 노드 구현
│   │   ├── routing.py         route, chat_answer
│   │   ├── planning.py        plan, gate          (+ 대상 시스템 해석 로직 신규)
│   │   ├── research.py        research, replan_check (+ 판별기 주입·다중 runbook)
│   │   ├── analysis.py        analysis
│   │   └── writing.py         write, validate, finalize (+ 수치 근거 검증)
│   ├── classifier.py        🟩 runbook 판별기 2종(keyword/rag) + 시스템 해석기
│   ├── rag/                 🟩 RAG 계층
│   │   ├── embedder.py        OpenAI 임베더 / 오프라인 Hash 임베더
│   │   ├── store.py           벡터 스토어(검색·디스크 캐시)
│   │   └── knowledge.py       runbook·시스템 카탈로그를 청크로 만들어 인덱싱
│   ├── systems.py           🟩 시스템 목록 단일 소스 (3곳 복붙 제거)
│   ├── keywords.py          🟩 fallback 라우팅 키워드 (3곳 복붙 제거 + 보강)
│   ├── fake_llm.py          🟨 결정론적 가짜 LLM (keywords 공유만)
│   ├── openai_llm.py        🟨 OpenAI 백엔드 (systems/keywords 공유, 프롬프트 1줄)
│   ├── ollama_llm.py        🟨 Ollama 백엔드 (위와 동일)
│   ├── mcp_client.py        🟦 MCP 다중 서버 클라이언트
│   ├── retry.py             🟦 일시적 오류 재시도
│   ├── session_store.py     🟦 SQLite 대화 저장소
│   └── supervisor_v1.py     🟦 구버전 선형 파이프라인 (롤백용)
│
├── data/
│   ├── monitoring.json      🟨 시스템별 모니터링 기준값 (+3개 시스템)
│   ├── systems.json         🟩 시스템 설명 카탈로그 (RAG 검색 대상)
│   └── runbooks/*.md        🟨 대응 매뉴얼 8종 (기존 6 + 신규 2, 전부 "주요 증상" 줄 추가)
│
├── tests/                   🟩 pytest 27개
├── eval/                    🟩 평가 데이터셋·실행기·비교표·벤치마크·결과
└── doc/presentation/        🟩 발표자료 생성기, 이 문서
```

## 2. 먼저 알아야 할 핵심 개념 5개

### ① LangGraph StateGraph — "상태(dict)를 들고 노드를 차례로 도는 기계"

- 그래프에는 **공유 상태(AgentState)** 가 하나 있어요. 모든 노드가 이걸 읽어요.
- 노드는 `async def node(state, deps) -> dict` 함수예요. 반환한 dict는 **바꾸고 싶은 필드만** 담은 **델타**이고, LangGraph가 이걸 상태에 덮어써요.
- 다음 노드는 엣지가 정해요. 고정 엣지( `plan → gate` )와 조건부 엣지( `route → chat_answer 또는 plan` )가 있어요.
- 이 프로젝트에는 reducer(누적 규칙)가 없어요. 그래서 `events` 처럼 쌓여야 하는 리스트는 **각 노드가 기존 리스트에 붙인 새 리스트 전체**를 반환해요. 코드에 `state["events"] + [event]` 패턴이 반복되는 이유예요.

### ② 의존성 주입(DI) — "노드는 필요한 도구를 인자로 받는다"

- LLM, MCP 클라이언트, 판별기 같은 외부 의존성은 전부 `SupervisorDeps` 하나에 담겨 있어요.
- `build_graph`에서 `partial(node_fn, deps=deps)`로 각 노드에 꽂아요. 노드 코드는 "deps가 진짜 OpenAI인지 가짜인지" 몰라도 돼요.
- 효과: LLM 백엔드 교체, keyword/rag 판별기 교체, 테스트용 가짜 MCP 주입이 **노드 코드 수정 없이** 가능해요.

### ③ MCP의 3가지 capability

| 종류 | 이 프로젝트에서 | 언제 쓰나 |
|---|---|---|
| **Tool** | `get_monitoring_status(system_name)` | 호출할 때마다 결과가 달라지는 **동작/조회** |
| **Resource** | `runbook://incident-response/{incident_type}` | URI로 식별되는 **정적 문서** |
| **Prompt** | `incident_analysis_prompt(system_name, symptom)` | 서버가 제공하는 **표준 지시문 템플릿** |

### ④ SSE(Server-Sent Events) — "서버가 한 줄씩 밀어주는 스트림"

- 응답 본문이 `data: {json}\n\n` 형태로 계속 흘러와요. 마지막엔 `data: [DONE]`이 와요.
- 브라우저의 `EventSource`가 줄 단위로 받아서, 진행 단계(⏳)와 답변 토큰을 실시간으로 그려요.

### ⑤ "모델을 믿지 말고 코드로 검증" 원칙 (베이스의 설계 철학)

- **gate**: 모델이 "분석 필요 없음"이라고 해 놓고 analysis task를 끼워 넣으면 코드가 지워요.
- **replan**: runbook이 안 모였으면 코드가 추가 조사를 시켜요.
- **validate**: 답변 형식이 안 맞으면 코드가 반려해요.
- 이번 개선도 같은 원칙을 따랐어요. RAG 검색 결과는 **메타데이터(incident_type)만** 쓰고, 문서 조회는 MCP가 해요. 시스템명이 없으면 **코드가 target을 확정**해요. 답변 수치는 **코드가 대조**해요.

---

## 3. 요청 하나 따라가기 — state가 실제로 이렇게 바뀐다

질문: `payments-service CPU 급증 장애 진단해줘` (fake LLM 기준 실제 출력. fake는 일부러 결함을 재현하도록 만들어져 있어요.)

| 실행된 노드 | 반환한 델타 (state에 덮어쓰는 값) | 해설 |
|---|---|---|
| route | `route="incident"`, `needs_analysis=false` | 장애 질의로 분류 |
| plan | `tasks=[task-0 payments-service, task-1 web-01, task-analysis]` | fake가 일부러 analysis task를 끼워 넣음(자기모순 재현) |
| gate | `tasks=[task-0, task-1]`, `gate_removed=1` | needs_analysis=false니 코드가 analysis 제거 |
| research | 두 task `done`, `collected=[monitoring, monitoring]`, `parallel_elapsed=0.35` | 모니터링 2건을 동시에 시작 |

| 실행된 노드 | 반환한 델타 (state에 덮어쓰는 값) | 해설 |
|---|---|---|
| replan_check | `tasks += [task-gap-runbook-0 runbook cpu-spike]`, `replan_rounds=1`, `incident_types=[cpu-spike(payments, 0.313)]` | runbook이 없음 → **RAG 판별** → 추가 조사 task 생성 |
| research | gap task `done`, `collected += [runbook]` | MCP Resource로 cpu-spike 본문 조회 |
| replan_check | `{}` (변경 없음, 이벤트만) | runbook 있음 → 충분 |
| analysis | `{}` | analysis task 없음 → 스킵 |
| write | `draft="재시작하면 됩니다."` | fake의 1차 초안은 일부러 부실함 |
| validate | `validation_attempts=1`, `validation_feedback="응답이 너무 짧거나…"` | 반려 → write로 되돌아감 |
| write | `draft="'…' 진단 결과: 1) 모니터링 … 2) 표준 절차 …"` | 피드백 반영 재작성 |
| validate | `validation_attempts=2`, `validation_feedback=null` | 합격 |
| finalize | `final=draft` | 최종 답변 확정 |

> 노드 하나하나가 "state 일부를 읽고 → 일을 하고 → 바뀐 필드만 돌려준다"는 걸 이 표로 설명하면 돼요.

## 4. 파일별 상세

### 4-1. `server.py` 🟨 — FastAPI 진입점

| 함수 | 하는 일 |
|---|---|
| `_build_llms()` | `LLM_BACKEND` (fake/ollama/openai)에 따라 **fast LLM**(분류·계획용, 싼 모델)과 **quality LLM**(답변 작성용, 비싼 모델)을 만들어요 |
| `_build_classifiers()` 🟩 | `INCIDENT_CLASSIFIER`가 `keyword`면 기존 판별기를 써요. `rag`(기본)면 임베더를 고르고 → `KnowledgeBase.build()`로 인덱스를 만든 뒤 → `RagIncidentClassifier`와 `SystemResolver`를 **같은 지식베이스로** 만들어요 |
| `lifespan()` | 서버 수명 동안 **딱 한 번** 실행돼요. MCP 연결 → LLM → 판별기 → `SupervisorDeps` 조립 → `build_graph(deps)` 컴파일 → `app.state`에 저장. 종료 시 MCP, DB를 정리해요 |
| `GET /health` | 상태 확인. `creation_count`가 1이면 싱글턴이 정상이에요. 판별기 종류, 임베더, 캐시 히트 여부도 보여줘요 |
| `_handle_chat()` | 세션 이력 로드 → `graph.ainvoke(초기 상태)` → 사용자/답변을 세션에 저장. `SUPERVISOR_VERSION=v1`이면 구버전 함수로 가요 |
| `POST /chat` | 한 번에 응답 + events + metadata(`_metadata`: 조사 수, 재시도, 비용, 판별된 runbook 등) |

| 함수 | 하는 일 |
| --- | --- |
| `GET /chat/stream` | `graph.astream(stream_mode=["updates","custom"])` 를 써요. **updates**는 노드가 끝날 때마다의 델타로, 새로 생긴 events만( `seen` 카운터로 구분) SSE로 내보내요. **custom**은 write/chat 노드가 `get_stream_writer()` 로 흘리는 답변 토큰( `AnswerToken` )이에요. 마지막에 `FinalAnswer` , `[DONE]` 이 와요 |

### 4-2. `agent/state.py` 🟩 — 공유 상태

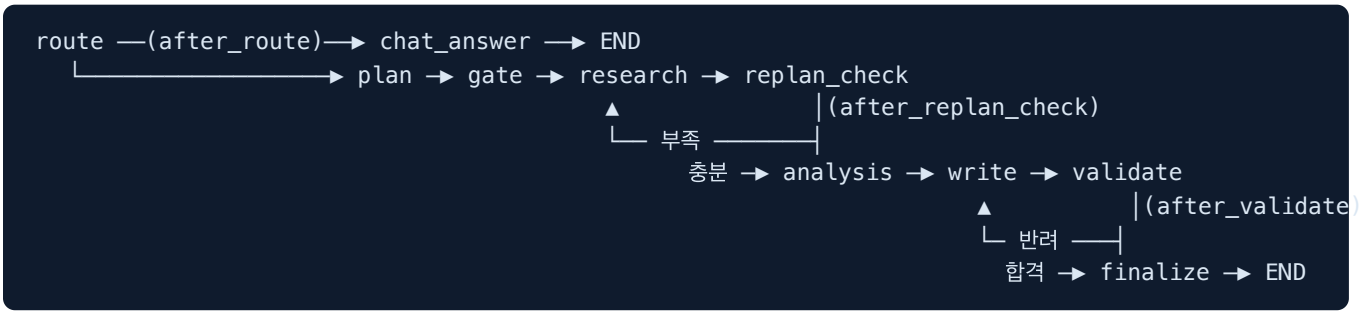
| 필드 | 쓰는 노드 → 읽는 노드 |
| --- | --- |
| `query` , `thread_id` , `history` | server(초기값) → route, plan, chat_answer, write |
| `route` , `needs_analysis` | route → after_route, gate |
| `resolved_system` 🟩 | plan → server metadata (시스템 해석 근거) |
| `tasks` | plan → gate → research ⇄ replan_check → analysis (할 일 목록, status pending/done) |
| `collected` | research, analysis → replan_check, write, validate (수집한 사실) |
| `replan_rounds` | replan_check (무한 루프 방지 카운터) |
| `incident_types` 🟩 | replan_check → metadata (판별 결과와 점수) |
| `retry_count` , `parallel_elapsed` | research → metadata |
| `draft` , `validation_attempts` , `validation_feedback` | write ⇄ validate |
| `final` | finalize / chat_answer → server |
| `cost_usd` , `events` | 모든 노드가 누적 |

`add_cost(state, usage)` 는 누적 비용에 이번 호출 비용을 더해 주는 헬퍼예요.

### 4-3. `agent/deps.py` 🟨

`SupervisorDeps` dataclass. 원래 필드는 `fast_llm, quality_llm, mcp, session_store, version` 이고, 추가한 필드는 `incident_classifier` , `system_resolver` , `grounding_check` 예요. 추가 필드는 **기본값이 있어서** 기존 코드와 테스트가 그대로 동작해요.

### 4-4. `agent/graph.py` 🟨 — 배선만



조건 함수 3개(`after_route`, `after_replan_check`, `after_validate`)는 각 노드 파일에 같이 있어요. **"분기 기준은 그 노드를 만든 사람이 제일 잘 안다"** 는 이유예요.

### 4-5. `agent/nodes/` 🟩 (구현 내용은 원본 + 개선)

**routing.py** - `route_node`: `deps.fast_llm.route(query)` → `{"route", "needs_analysis"}`. OpenAI 백엔드는 JSON 모드로 받고, 파싱에 실패하면 `keywords.looks_like_incident`로 fallback해요. - `chat_answer_node`: 잡담 경로. 대화 이력 최근 10개를 넣어 답해요(이름 기억 데모). 토큰을 `writer({"type":"AnswerToken"...})`로 흘려요.

**planning.py** - `_resolve_targets` 🟩: **조사 대상을 정하는 4단계**예요. 1. 등록 시스템명 있음 → 그대로(모델 계획 존중, 강제 없음) 2. 미등록 시스템명(정규식 `_SYSTEM_NAME_RE`) → 그 이름으로 강제 조사 3. 이름 없음 → `system_resolver.resolve(query)` (카탈로그 RAG) 4. 실패 → `DEFAULT_SYSTEM` - `plan_node`: MCP에서 진단 절차 Prompt를 받아(`find_prompt` → `get_prompt`) fast LLM에게 task 목록을 만들게 해요. 2·3단계로 확정된 대상이 있으면 `_override_research_targets`가 모델이 정한 research task를 **코드가 교체**해요. - `gate_node`: `needs_analysis=False`면 analysis task를 제거해요.

**research.py** - `research_node`: `pending` 상태인 research task들을 `asyncio.gather`로 동시에 실행해요. monitoring은 `with_retry(mcp.call_tool)`, runbook은 `find_resource_template`로 URI 템플릿을 찾아 `read_resource`로 가져와요. 결과는 `collected`에 쌓고 task는 `done`으로 바꿔요. - 참고: MCP 클라이언트 내부 `asyncio.Lock` 때문에 실제 MCP 호출은 하나씩 나가요. - `replan_check_node` 🟨: runbook이 이미 있거나 `replan_rounds>=1`이면 "충분"으로 판정해요. 아니면 `deps.incident_classifier.classify()` → 판정된 유형마다 `task-gap-runbook-i`를 만들어요. 판정이 비면(미등록 시스템뿐) 🟩 runbook 없이 바로 충분 처리해요.

**analysis.py** - gate가 살려둔 analysis task가 있을 때만 `fast_llm.analyze(collected)`로 여러 시스템을 비교 분석해요(예: "공통 원인 분석해줘").

**writing.py** - `write_node`: `quality_llm.write(query, collected, attempt, feedback)`. 재작성이면 직전 반려 사유를 프롬프트에 붙여요. - `ungrounded_percentages` 🟩: 수집된 사실 전체를 문자열로 만들어 숫자를 모은 뒤, 답변의 `N%` 중 그 목록에 없는 것을 반환해요. - `validate_node` 🟨: ① 30자 이상이고 `1)`가 있는지 ② (grounding_check면) 근거 없는 수치가 없는지 확인해요. 문제가 있으면 사유를 합쳐 `validation_feedback`으로 넘기고, 3회째엔 통과시켜요. - `finalize_node`: `final = draft`.

### 4-6. `agent/classifier.py` 🟩 — 어떤 runbook을 가져올까

| 클래스 | 동작 |
|---|---|
| `KeywordIncidentClassifier` | **개선 전 로직 그대로.** 모든 모니터링의 status와 alerts를 한 문자열로 합치고, `INCIDENT_TYPE_RULES`를 위에서부터 훑어 처음 걸리는 유형 1개를 반환. 없으면 `db-down` |
| `RagIncidentClassifier` | ① 조사 대상이 전부 미등록이면 `[]` ② 알림이 있는 시스템마다 "알림1; 알림2" 문장을 만들고(없으면 질문 문장) ③ `kb.search(문장들, kind="runbook", k=1)` 한 번으로 batch 검색 ④ 시스템별 top-1 청크의 `metadata.incident_type` → 중복 제거 → 최대 3개 |
| `SystemResolver` | `kb.search([query], kind="system", k=1)` → `{"system", "score"}` (점수가 `min_score` 미만이면 None) |

반환 형식은 모두 `(판정 리스트, usage)`예요. usage에 비용·지연(ms)이 들어 있어서 노드가 비용을 누적하고 이벤트에 지연을 기록해요.

### 4-7. `agent/rag/` 🟩 — RAG 계층

**embedder.py** - `OpenAIEmbedder.embed(texts)` : `/v1/embeddings` 에 텍스트 리스트를 **한 번에** 보내고, 응답 순서(index)대로 벡터를 정렬한 뒤 L2 정규화해요. 비용은 실측 토큰 수 × 단가예요. - `HashEmbedder` : 문자 2~3그램을 md5 해시해서 512칸 벡터에 세는 방식이에요. 네트워크와 과금 없이 결정론적이라 테스트용으로 써요. - 정규화하는 이유: 길이 1 벡터끼리는 **내적 = 코사인 유사도**라서 검색이 단순해져요.

**store.py** - `Document(id, text, metadata)` , `Hit(doc, score)` - `VectorStore.search(query_vec, k, where)` : 모든 문서와 내적 → metadata 필터 → 점수순 상위 k개. - `save/load(path, key)` : JSON 파일 캐시. key가 다르면 `None` 을 반환해서 재구축하게 해요. - `corpus_hash(docs)` : 모든 청크의 id+본문 SHA-256 앞 16자리.

**knowledge.py** - `load_runbook_documents` : runbook 하나 → 요약 청크 1개( `제목\n주요 증상` ) + 번호 절차마다 청크 1개. metadata는 `{kind:"runbook", incident_type, section}` . - `load_system_documents` : `systems.json` 항목마다 `"이름: 설명"` 청크. metadata는 `{kind:"system", system}` . - `KnowledgeBase.build()` : 청크 생성 → 캐시키 `"임베더명:코퍼스해시"` → 캐시가 있으면 로드(API 0회), 없으면 전부 임베딩 후 저장. - `KnowledgeBase.search(texts, kind, k)` : 텍스트들을 1회 임베딩 → 텍스트마다 `kind` 필터 검색.

### 4-8. `agent/systems.py` · `agent/keywords.py` 🟩

- `KNOWN_SYSTEMS` 는 `monitoring.json` 의 키, `SYSTEM_CATALOG` 는 `systems.json` (등록된 시스템만), `DEFAULT_SYSTEM` 은 환경변수(기본 auth-service)예요.
- `find_mentioned_systems(query)` : 질문에 나온 시스템을 **등장 순서대로** 반환해요.
- `guess_target(s)` : LLM 계획이 깨졌을 때 복구용이에요(원래 백엔드 파일마다 있던 함수).
- `INCIDENT_HINT_RE` : 기존 7개 단어에 "느려·치솟·타임아웃·에러·죽·5xx…"를 추가했어요. LLM 라우터가 실패했을 때와 fake 모드에서만 써요.

### 4-9. LLM 백엔드 3종 ( `fake_llm.py` / `openai_llm.py` / `ollama_llm.py` ) 🟨

**공통 인터페이스** — 그래프는 이 메서드 이름만 알아요.

| 메서드 | 담당 | 반환 |
|---|---|---|
| `route(query)` | fast | `({"route","needs_analysis"}, usage)` |
| `plan(query, procedure)` | fast | `({"needs_analysis","tasks":[...]}, usage)` |
| `analyze(collected)` | fast | `(요약문, usage)` |
| `chat(query, history, on_token)` | fast | `(답변, usage)` async |
| `write(query, collected, attempt, feedback, on_token)` | quality | `(초안, usage)` async |

- **openai**: Chat Completions, `temperature=0` , route/plan은 `response_format=json_object` (JSON 모드)를 써요. JSON 모드는 "JSON 형식"만 보장하고 스키마는 보장하지 않아서, `plan()` 뒤에 **코드 레벨 정규화**를 해요. 알 수 없는 target은 질문에서 다시 찾고, 중복은 제거하고, research가 0개면 질문에 나온 시스템 전부로 복구해요.
- **ollama**: 로컬 모델이에요. `format=<JSON Schema>` 로 스키마까지 강제하고, 답변은 NDJSON 스트림으로 토큰 단위 전송해요.

- **fake**: 결정론 스텁이에요. 수업 데모용으로 **일부러 결함을 재현**해요. ① plan이 analysis task를 끼워 넣음 → gate 데모 ② web-01을 항상 추가 → 병렬 데모 ③ 1차 초안이 "재시작하면 됩니다." → validate 데모.
- 비용은 모두 `토큰 수 × usd_per_1k` 하나의 단가로 계산한 추정치예요(실제 청구액과 다를 수 있음).
- 수정한 부분: 시스템 목록과 키워드를 공통 모듈에서 import하게 바꿨고, plan 프롬프트의 "언급 없으면 auth-service" 문구를 "표준 절차의 '시스템' 항목을 조사하라"로 바꿨어요.

### 4-10. MCP 계층 🟦

**mcp_server.py** (fastmcp) - `get_monitoring_status` : `monitoring.json` 기준값에 무작위 jitter(-3~+8)를 더해 "실시간처럼" 반환해요. 모르는 시스템이면 `status="unknown"` 과 not found 알림을 돌려줘요. - `get_incident_runbook` : `data/runbooks/{type}.md` 본문을 반환해요(없으면 Not Found 문서). - `incident_analysis_prompt` : "시스템/증상 + 3단계 진단 절차" 텍스트 템플릿이에요.

**mcp_client.py** - `start()` : 서버 스크립트마다 `stdio_client` subprocess와 `ClientSession` 을 열고 `initialize()` 해요. `AsyncExitStack` 으로 한꺼번에 정리해요. - `find_prompt / find_resource_template` : 연결된 서버를 순서대로 `list_*` 해서 **이름을 가진 서버를 찾은 뒤** 요청해요(어느 서버에 있는지 하드코딩하지 않음). - `call_tool` : 프로세스 첫 모니터링 호출에서 **일부러** `TimeoutError` 를 내요(재시도 데모). - `_attr` : mcp SDK 버전에 따라 필드명이 camelCase/snake_case로 갈려서 둘 다 시도해요. - `self._lock` : stdio 세션 하나를 여러 코루틴이 동시에 쓰지 않게 직렬화해요.

**mcp_server_docs.py**: API 문서 Resource만 있는 decoy예요. "서버가 여러 개일 때 찾아서 요청하는가"를 보여주려는 용도예요.

### 4-11. 기타 원본 모듈 🟦

- **retry.py** `with_retry` : `asyncio.TimeoutError` 만 0.3초 → 0.9초로 최대 3회 재시도하고, `(결과, 재시도 횟수)` 를 반환해요. 다른 예외는 즉시 전파해요.
- **session_store.py**: `messages(thread_id, role, content, created)` 테이블 하나예요. 파일 DB라 서버를 재시작해도 이름을 기억해요.
- **supervisor_v1.py**: 라우팅·검증이 없는 구버전이에요. `SUPERVISOR_VERSION=v1` 로 재배포 없이 롤백하는 데모용이에요.
- **web_demo_server.py**: `/` 는 HTML 파일, `/chat/stream` 은 8100의 스트림을 바이트 그대로 중계해요.
- **web_chat.html**: Vue 3(CDN)로 만들어져 있어요. `EventSource` 로 이벤트를 받아 `STEP_LABELS` 로 진행 단계 문구를 만들고, `AnswerToken` 은 말풍선에 이어 붙여요. `attempt` 가 바뀌면(재작성) 새 말풍선을 만들어요.

### 4-12. 데이터 🟨

- `monitoring.json` : `{시스템: {status, cpu_base, memory_base, alerts[]}}` 형식이에요. 신규 order/search/notification-service는 일부러 **키워드 규칙에 없는 표현**("HTTP 5xx", "OOM killed", "TLS handshake")을 썼어요.
- `runbooks/*.md` : `# 제목` / `- incident_type:` / `- 주요 증상:` / `1. … 5.` 번호 절차 형식이에요. 파일명이 곧 incident_type이에요.
- `systems.json` : `{시스템: "한 줄 설명"}` 이에요. 시스템명 없는 질문을 해석할 때 검색 대상이 돼요.

### 4-13. `tests/` 🟩

- `conftest.py`의 `InMemoryMCP` : MCPClient와 **같은 메서드 이름**으로 data/ 파일을 직접 읽어요(jitter 없음). `StubClassifier`는 정해진 유형을 반환해요. `make_deps`는 필요한 의존성만 바꿔 끼우는 팩토리예요.
- `test_systems_keywords.py` (5) · `test_rag.py` (10) · `test_nodes.py` (10) · `test_graph.py` (2)
- 실행: `.venv/bin/python3 -m pytest -q` (asyncio 모드는 `pytest.ini`에서 auto)

### 4-14. `eval/` 🟩

- `dataset.jsonl` : `{id, set, query, route, targets, types, allowed_types}`가 24줄이에요.
- `run_eval.py` : `--project`로 원본 스냅샷도 같은 방식으로 import해요. `lifespan` 안에서 `_handle_chat`을 직접 호출해 **최종 state**를 얻고 → `score_row`가 tasks에서 "조사한 시스템"과 "가져온 runbook"을 뽑아 라벨과 비교해요. `--judge`면 gpt-4o-mini가 1~5점으로 채점해요. 결과는 `results/<label>.json`에 저장해요.
- `compare.py` : 여러 결과를 나란히 놓은 마크다운 표를 만들어요.
- `bench_classifier.py` : 그래프 없이 판별기만 반복 실행해서 지연과 비용을 따로 재요.

---

## 5. 설정(환경변수) 한눈에

| 변수 | 기본값 | 의미 |
|---|---|---|
| `LLM_BACKEND` | fake | fake / ollama / openai |
| `SUPERVISOR_VERSION` | v2 | v1이면 구버전 파이프라인 |
| `INCIDENT_CLASSIFIER` 🟩 | rag | keyword면 개선 전 판별기 |
| `EMBEDDING_BACKEND` 🟩 | openai 백엔드면 openai, 아니면 hash | 임베더 선택 |
| `RAG_MIN_SCORE` / `RESOLVER_MIN_SCORE` 🟩 | 0.0 | 이 점수 미만이면 판정 버림 |
| `DEFAULT_TARGET_SYSTEM` 🟩 | auth-service | 최후 기본 조사 대상 |
| `VALIDATE_GROUNDING` 🟩 | 1 | 0이면 수치 근거 검사 끔 |
| `OPENAI_FAST_MODEL` / `OPENAI_QUALITY_MODEL` | gpt-4o-mini / gpt-4o | |
| `LANGCHAIN_TRACING_V2` | (.env) | LangSmith 추적 on/off |

---

## 6. 코드에 대해 스스로 확인해볼 질문 (답은 위에 다 있음)

1. 노드는 state 전체를 반환하나, 일부만 반환하나? 왜 `events`는 전체 리스트를 반환하나?
2. `partial(fn, deps=deps)`를 안 쓰면 노드가 deps를 어떻게 얻어야 했을까?
3. RAG 판별에서 검색 결과의 **본문**은 어디에 쓰이나? (답: 안 쓰인다 — metadata만 쓰고 본문은 MCP Resource로 따로 가져옴)
4. 서버를 두 번째로 켤 때 임베딩 API가 호출되지 않는 이유는?
5. research ⇄ replan_check 루프가 무한히 돌 수 없는 이유는?
6. billing-service 질문은 4단계 중 어디로 가고, 최종적으로 runbook은 몇 개가 붙나? (답: 2단계, 0개)
7. `/chat/stream`에서 답변 토큰과 진행 이벤트는 각각 어떤 stream_mode로 오나?